# MASTER PROMPT — AICTE FDR MANAGEMENT PORTAL | SIH PROJECT

Act as a senior full-stack engineer, application security architect, UI/UX designer, and software tester. Build a complete, responsive, secure, production-minded web application prototype for a Smart Fixed Deposit Receipt (FDR) Management System, designed around the AICTE institutional FDR management problem statement for a Smart India Hackathon project.

The application must have a professional government-portal design, functional workflows, persistent database storage, role-based access control, robust input validation, secure document handling, audit logging, and automated maturity monitoring.

Do not build only a static dashboard or a frontend mockup. Build a working end-to-end application with a frontend, backend, database integration, authentication, authorization, and tested business logic.

## 1. PROJECT OBJECTIVE

Create a centralized digital platform that helps educational institutions manage their FDR records and supporting documents throughout their lifecycle.

The platform should enable authorized users to:
- Register and manage institutional FDR records.
- Upload FDR receipts and supporting documents.
- Submit records for verification.
- Review, approve, reject, or return records for corrections.
- Track FDR maturity dates, renewals, release requests, and application statuses.
- Identify duplicate records, missing documents, and approaching deadlines.
- Maintain a transparent, searchable, auditable history of actions.

Important: This is an independent hackathon prototype, not an officially affiliated AICTE system. Do not claim that the application is connected to AICTE, banks, government databases, or payment systems unless a real, authorized integration has been configured and verified.

## 2. RECOMMENDED TECHNOLOGY STACK

Use the following stack unless a genuine technical incompatibility requires a documented alternative.

Frontend:
- React with TypeScript.
- Vite.
- Tailwind CSS.
- React Router.
- React Hook Form with Zod validation.
- Recharts for dashboard visualizations.
- Accessible components and responsive layouts.

Backend:
- Node.js and Express.js.
- TypeScript.
- REST APIs.

- Server-side validation using Zod.
- Centralized error handling and structured logging.

Database:
- MongoDB with Mongoose.
- Proper schema validation, indexes, timestamps, and unique constraints where appropriate.

Authentication and security:
- Use a mature authentication library or a properly implemented server-side authentication system.
- Hash passwords with Argon2id or another appropriate modern password-hashing algorithm.
- Prefer secure, HttpOnly, SameSite cookies for browser sessions.
- Implement CSRF protection where applicable.
- Add login rate limiting, secure session expiration, and logout/session invalidation.
- Never store passwords, authentication tokens, or session identifiers in localStorage.

File storage:
- Use private object storage or an equivalent secure file-storage service.
- If external storage is unavailable, provide a clearly identified local development adapter.
- Never expose uploaded documents through public, predictable URLs.

Configuration:
- Keep secrets and database credentials in environment variables.
- Include a safe `.env.example` containing placeholders only.
- Never place secret keys in frontend code or commit real credentials.

If any external service is unavailable, implement a clearly documented development adapter. Do not pretend a simulated integration is real.

## 3. USER ROLES AND PERMISSIONS

Implement server-enforced role-based access control (RBAC).

### A. Institution User
- Access only their institution's authorized records.
- Create and edit draft FDR records.
- Upload supporting documents.
- Submit records for verification.
- View verification comments and correction requests.
- Resubmit corrected records.
- Request renewal or release, subject to the configured workflow.
- Track deadlines, alerts, and application history.
- Export only authorized institutional data.

### B. Verification Officer
- View records assigned to them or within their authorized scope.
- Review FDR details and documents.
- Check required fields and document completeness.
- Approve, reject, or return submissions for correction.
- Add mandatory comments for rejection or correction.
- Record the verification outcome and supporting rationale.
- Review the audit history of authorized records.

### C. System Administrator
- Manage authorized users, institution accounts, role assignments, and account status.
- Configure document requirements, alert intervals, and workflow settings.
- View system-wide operational metrics only within the administrator's permitted scope.
- Review security events and audit records.
- Manage reviewer assignments and escalation rules.

An administrator must not automatically gain permission to perform every financial approval. Separate administrative account management from financial verification authority.

Enforce least privilege, deny access by default, and verify authorization on every protected backend endpoint. Hiding a button in the frontend is not sufficient security.

For the hackathon demo, seed separate demo accounts for each role. Clearly distinguish them from real accounts.

## 4. REQUIRED APPLICATION MODULES

### Module 1: Authentication and Account Management
- Login and logout.
- Institution onboarding or invitation-based registration.
- Password reset through a secure token-based flow, or a clearly marked development-only adapter.
- Role-based landing pages.
- Session expiration and session revocation.
- Account activation and deactivation.
- Optional multi-factor authentication architecture for privileged users.
- Clear error messages that do not reveal whether an account exists.

Do not provide a public registration route that allows users to assign themselves privileged roles.

### Module 2: Institution Dashboard
Display:
- Total FDR records.
- Draft records.
- Pending verification.
- Records requiring correction.
- Verified and active FDRs.
- Upcoming maturity dates.
- Matured FDRs.
- Renewal requests.
- Release requests.
- Missing-document alerts.

Include:
- Searchable tables.
- Filters by status, institution, bank, date, and amount.
- Pagination.
- Sorting.
- Date-range filters.
- Visual charts for status distribution and maturity trends.

- Clear empty states and loading indicators.

Every statistic must be calculated from authorized database records, not hardcoded numbers.

### Module 3: FDR Registration and Management

Create a validated form with the following fields:

- Internal FDR record ID.
- Institution ID and institution name.
- Bank name.
- Branch name, where required.
- FDR number or reference.
- Deposit amount in INR.
- Issue date.
- Maturity date.
- Interest rate, if applicable and required.
- Deposit purpose or category.
- Issuing institution or account reference, only where necessary.
- Supporting document uploads.
- Additional remarks.
- Creation and last-update timestamps.

Requirements:
- Validate required fields on both the client and server.
- Validate date consistency.
- Reject invalid or negative deposit amounts.
- Prevent unauthorized changes after submission.
- Maintain a distinction between an internal record ID and a bank-issued FDR reference.
- Enforce unique constraints according to the configured business rules.
- Never assume that a matching FDR number alone proves fraud or duplication.

Do not request or store bank login credentials, PINs, OTPs, CVVs, or unnecessary personal financial information.

### Module 4: Secure Document Management

Allow authorized users to upload:
- FDR receipt.
- Supporting bank confirmation, if required.
- Renewal documentation.
- Release-related documentation.
- Other configurable supporting documents.

Security requirements:
- Allow only explicitly permitted file types.
- Check actual file signatures, not only file extensions or browser-provided MIME types.
- Enforce configurable file-size limits.
- Generate randomized storage names.
- Store files in private storage.
- Prevent path traversal and arbitrary file access.
- Scan uploads for malware where supported; otherwise mark scanning as unavailable or pending.
- Do not expose unscanned files to reviewers.

- Use short-lived, authorization-checked download links.
- Record document metadata, uploader, upload time, scan status, and version.
- Prevent executable files and unsafe content from being served directly.
- Apply access checks whenever a document is viewed or downloaded.
- Keep document access events in the audit trail.

If OCR is added, treat extracted text as untrusted input. OCR results must not automatically approve an FDR or overwrite verified records without authorized confirmation.

### Module 5: Verification and Approval Workflow

Implement this workflow:

Draft → Submitted → Under Review → Approved / Rejected / Correction Required.

A correction workflow should support:

Correction Required → Resubmitted → Under Review → Final Decision.

Implement a separate lifecycle state for the FDR itself, such as:

Active → Maturity Approaching → Matured → Renewal Requested / Release Requested → Closed, where applicable.

Do not mix application verification status with the financial lifecycle status.

Requirements:
- Only authorized reviewers can make verification decisions.
- Record who performed the action and when.
- Require a reason for rejection or correction.
- Preserve previous decisions and comments.
- Notify the institution when the status changes.
- Prevent unauthorized state transitions.
- Prevent duplicate submissions and accidental repeated actions.
- Use server-side state-transition validation.
- Use database transactions where supported and appropriate to protect related updates.

Approval in this application represents a workflow decision only. It does not independently prove that the bank issued a genuine FDR.

### Module 6: Smart Maturity Monitoring

Calculate maturity status from the recorded maturity date and configured rules.

Default alert intervals:
- 30 days before maturity.
- 15 days before maturity.
- 7 days before maturity.
- On the maturity date.
- After maturity, if the record remains unresolved.

Provide:

- Upcoming maturity dashboard.
- Overdue and unresolved records.
- Renewal reminders.
- Release-request reminders.
- Reviewer escalation alerts.
- In-app notification history.

Requirements:
- Use a consistent configured timezone.
- Calculate deadlines on the server.
- Prevent repeated notifications for the same event.
- Record notification delivery status.
- Use a scheduled background job or equivalent reliable scheduler.
- Do not depend on the user keeping the dashboard open.

Do not automatically renew, release, transfer, or withdraw money when a maturity date is reached.

### Module 7: Duplicate Detection and Data Validation

Implement a rule-based duplicate detection module.

Compare configurable combinations of:
- FDR reference.
- Bank identifier.
- Institution identifier.
- Deposit amount.
- Issue date.
- Maturity date.

Requirements:
- Normalize values before comparison.
- Use appropriate database indexes.
- Flag possible duplicates for human review.
- Explain which fields matched.
- Avoid automatically rejecting a record solely because it resembles another record.
- Preserve the review outcome and reviewer comments.

Keep the duplicate-detection logic modular so that OCR or additional anomaly detection can be introduced later.

### Module 8: Renewal and Release Requests

Provide separate request workflows for:
- FDR renewal.
- FDR release or closure.

Allow the institution to submit a request with the required supporting documents.

Track:
- Request type.
- Linked FDR record.
- Request date.
- Submitted documents.
- Reviewer comments.
- Approval status.

- Outstanding requirements.
- Final recorded outcome.

All financial actions must remain outside the application unless an authorized, properly secured integration is explicitly implemented.

For the hackathon prototype, simulate these workflows without transferring real money.

### Module 9: Audit Trail and Security Monitoring

Create an append-only audit log for important events, including:
- Login successes and failures.
- Account and role changes.
- FDR creation and modification.
- Document upload, replacement, viewing, and download.
- Submission and resubmission.
- Verification decisions.
- Renewal and release requests.
- Administrative configuration changes.
- Export operations.
- Authorization failures.

Record:
- Event type.
- Actor ID.
- Target record ID.
- Timestamp.
- Outcome.
- Relevant change summary.
- Correlation or request ID.

Never log passwords, session tokens, OTPs, secret keys, or complete sensitive document contents.

Restrict audit-log access. Ordinary users must not be able to edit or delete audit entries. Use append-only application permissions and stronger tamper-evidence controls where feasible.

### Module 10: Reporting and Export

Provide authorized reports for:
- FDR status summaries.
- Upcoming maturity dates.
- Pending verification.
- Missing documents.
- Renewal and release requests.
- Duplicate-review cases.
- Verification history.

Support CSV export and print-friendly reports.

Requirements:
- Enforce the user's data-access permissions before generating a report.
- Escape CSV values to prevent spreadsheet formula injection.
- Avoid exposing unnecessary financial or personal information.
- Record export events.
- Apply pagination or controlled background processing to large exports.

## 5. SECURITY REQUIREMENTS — NON-NEGOTIABLE

Apply secure development practices throughout the application.

### Authentication
- Use secure password hashing.
- Use secure, expiring sessions.
- Configure appropriate cookie flags.
- Implement rate limiting and account lockout or progressive throttling where appropriate.
- Use secure password-reset tokens with expiry and single-use validation.
- Revoke sessions after password changes or account deactivation where appropriate.

### Authorization
- Enforce RBAC on the backend.
- Prevent cross-institution data access.
- Verify object ownership and reviewer assignment.
- Protect every API endpoint and file operation.
- Prevent users from escalating their own privileges.
- Keep sensitive actions behind explicit authorization checks.

### Web application security
- Use HTTPS in deployment.
- Configure Helmet and a restrictive Content Security Policy.
- Use explicit CORS allowlists.
- Protect against CSRF where cookie-based authentication requires it.
- Use parameterized database operations and strict schema validation.
- Prevent NoSQL injection, XSS, IDOR/BOLA, mass assignment, and unsafe deserialization.
- Validate and normalize all untrusted input.
- Avoid leaking stack traces or internal system details.
- Implement request-size limits, sensible timeouts, and rate limits.
- Do not use client-provided roles or status values as trusted authorization decisions.

### Secrets and privacy
- Never hardcode credentials or API keys.
- Never expose server secrets through frontend environment variables.
- Never commit `.env` files containing secrets.
- Minimize collection and retention of sensitive information.
- Use fictional records for demos and testing.
- Provide appropriate data-retention and deletion policies.
- Do not upload confidential documents to external AI services without explicit authorization.

### Database and infrastructure
- Use least-privilege database credentials.
- Validate database access configuration.
- Enforce appropriate indexes and constraints.
- Configure backups and recovery procedures for real deployments.
- Separate development, testing, and production configurations.
- Avoid publicly accessible database instances.
- Use controlled error handling and structured security logs.

### Financial integrity

- Never claim that an uploaded receipt has been verified by a bank unless a trusted verification integration actually confirms it.
- Treat uploaded documents as evidence requiring appropriate review.
- Separate record verification from bank-level authenticity verification.
- Do not implement real payments, deposits, transfers, or withdrawals in the demo.
- Clearly label simulated bank responses and demo data.
- Require authorized human approval for consequential workflow decisions.

## 6. USER INTERFACE AND DESIGN

Design a professional, clean, trustworthy government-service dashboard.

Visual style:
- White and light-grey backgrounds.
- Deep navy or blue primary color.
- Restrained teal or green for successful states.
- Amber for warnings.
- Red for errors and rejected states.
- Readable typography.
- Consistent spacing and accessible contrast.
- Clean cards, tables, forms, and status indicators.
- Avoid excessive gradients, unnecessary animations, oversized illustrations, and decorative clutter.

Layout:
- Responsive sidebar on desktop.
- Collapsible navigation on smaller screens.
- Top bar with page title, notifications, and profile menu.
- Breadcrumbs where useful.
- Responsive tables or mobile-friendly record cards.
- Clear loading, empty, success, and error states.
- Confirmation dialogs for consequential actions.

Main navigation:
1. Dashboard
2. FDR Records
3. Add FDR
4. Verification Queue
5. Maturity Calendar
6. Renewal and Release Requests
7. Documents
8. Reports
9. Audit Logs
10. User Management
11. Settings

Show only the navigation items permitted for the signed-in user's role.

The application must be keyboard-accessible, mobile-responsive, and usable with assistive technologies.

## 7. DASHBOARD DETAILS

Institution dashboard:
- Total FDRs.
- Active FDRs.
- Pending submissions.

- Correction requests.
- Upcoming maturity.
- Renewal and release request status.
- Recent activity.
- Alerts requiring attention.

Verification officer dashboard:
- Assigned verification queue.
- Pending review count.
- Records awaiting correction or resubmission.
- Upcoming maturity cases requiring attention.
- Recently completed reviews.
- Review turnaround indicators, where sufficient data exists.

Administrator dashboard:
- Institution and account statistics.
- Overall workflow status.
- Unassigned review cases.
- System notification status.
- Security events requiring attention.
- Operational reports within authorized permissions.

Every card, chart, filter, search field, and action must work with real application data or clearly labelled demo data. Do not leave decorative controls that appear functional but do nothing.

## 8. DATABASE DESIGN

Create appropriately related collections or equivalent tables for:

- Users
- Institutions
- FDRRecords
- Documents
- VerificationReviews
- RenewalRequests
- ReleaseRequests
- Notifications
- AuditLogs
- SystemSettings
- Sessions, if managed by the application

Include:
- Unique identifiers.
- Created and updated timestamps.
- Institution ownership.
- Appropriate status enums.
- Validation rules.
- Required indexes.
- Referential integrity checks.
- Soft deletion or archival rules where appropriate.
- Explicit handling of concurrent updates.

Do not store duplicate copies of the same sensitive information unnecessarily.

Create database migrations or initialization scripts, along with a safe demo-seeding script.

The demo-seeding process must never create publicly usable privileged credentials in a production environment.

## 9. BACKEND API DESIGN

Design documented REST endpoints with authorization and validation.

Examples:

Authentication:
- POST /api/auth/login
- POST /api/auth/logout
- GET /api/auth/me

Institutions:
- GET /api/institutions
- GET /api/institutions/:id

FDR records:
- GET /api/fdr
- POST /api/fdr
- GET /api/fdr/:id
- PATCH /api/fdr/:id
- POST /api/fdr/:id/submit

Verification:
- GET /api/reviews
- POST /api/fdr/:id/request-correction
- POST /api/fdr/:id/approve
- POST /api/fdr/:id/reject

Lifecycle:
- POST /api/fdr/:id/renewal-requests
- POST /api/fdr/:id/release-requests

Documents:
- POST /api/fdr/:id/documents
- GET /api/documents/:id/download

Reporting:
- GET /api/reports/fdr-summary
- GET /api/reports/maturity
- GET /api/reports/pending-verification

These routes are proposed examples, not a substitute for a complete API design. Add the remaining endpoints for institutions, notifications, user management, audit events, and configuration.

For every endpoint:
- Define the accepted request schema.
- Validate the user's identity and permissions.
- Verify ownership and record assignment.
- Validate the current workflow state.
- Return consistent HTTP status codes.
- Avoid exposing internal errors or unauthorized data.
- Add tests for success and failure cases.

Use safe pagination, filtering, sorting, and query limits.

## 10. OPTIONAL SMART FEATURES

After the core application is working, add the following features in priority order:

1. Smart maturity reminders.
2. Missing-document detection.
3. Duplicate FDR flagging.
4. OCR-assisted receipt data extraction.
5. Smart next-step recommendations.
6. Advanced analytics.

If using OCR or AI:
- Make it an optional, modular service.
- Validate all extracted values.
- Require human confirmation before changing verified financial records.
- Treat model output and uploaded document text as untrusted.
- Never allow document instructions or AI-generated text to bypass access controls.
- Do not send confidential documents to a third-party AI provider by default.
- Clearly identify automated suggestions as suggestions rather than verified facts.

Do not add a chatbot merely for decoration. Every AI feature must solve a specific problem and be demonstrable.

## 11. TESTING REQUIREMENTS

Create automated tests for the core functionality.

Authentication tests:
- Valid and invalid login.
- Expired sessions.
- Unauthorized role access.
- Session invalidation.

Authorization tests:
- Institution A cannot view or modify Institution B's FDRs.
- Institution users cannot approve their own submissions unless an explicitly authorized policy permits it.
- Reviewers cannot access unassigned records.
- Ordinary users cannot modify roles or audit logs.

FDR workflow tests:
- Create a valid record.
- Reject invalid amounts and dates.
- Submit a draft.
- Return a record for correction.
- Resubmit and approve a record.
- Prevent invalid status transitions.
- Flag duplicate records for review.

Document tests:
- Reject disallowed file types and oversized files.
- Deny unauthorized document downloads.

- Verify file-access logging.
- Prevent unscanned documents from becoming available for review.

Maturity tests:
- Trigger the configured alert windows.
- Prevent duplicate alerts.
- Handle matured and unresolved records correctly.
- Confirm that maturity never initiates an automatic financial transaction.

Security tests:
- Test server-side input validation.
- Test object-level authorization.
- Test rate limiting.
- Check for sensitive information in error messages and logs.
- Test CSV export safety.
- Run dependency and static security checks where available.

Fix discovered issues rather than merely documenting them.

## 12. DEVELOPMENT AND IMPLEMENTATION ORDER

Build the application in the following order:

Phase 1: Project foundation
- Create the frontend and backend project structure.
- Configure TypeScript, environment variables, linting, and error handling.
- Establish the database connection.
- Create schemas and indexes.
- Implement a safe demo-data seeder.

Phase 2: Authentication and authorization
- Implement login and logout.
- Create role-based access.
- Add institution ownership checks.
- Test forbidden access before proceeding.

Phase 3: Core FDR workflow
- Build the FDR registration form.
- Persist records in the database.
- Add record listing, search, filtering, and detail views.
- Implement submission and verification workflows.
- Add correction and resubmission handling.

Phase 4: Document management
- Add secure upload and download.
- Implement document validation and metadata.
- Add document versioning and access logs.

Phase 5: Monitoring and reporting
- Add maturity calculations and scheduled alerts.
- Add duplicate detection.
- Implement renewal and release requests.
- Add reports and CSV export.

Phase 6: Audit and security hardening
- Complete audit logging.

```
- Test authorization boundaries.
- Review session management, uploads, and API security.
- Fix security and functional defects.

Phase 7: Final demo
- Add a polished, responsive interface.
- Seed fictional demo data.
- Verify the complete user journey.
- Document setup and deployment.
- Prepare a short, reproducible demonstration.

Do not skip earlier phases to add advanced AI features.

## 13. EXPECTED PROJECT STRUCTURE

Use a maintainable structure similar to:

frontend/
  src/
    components/
    layouts/
    pages/
    routes/
    services/
    hooks/
    schemas/
    types/

backend/
  src/
    config/
    middleware/
    models/
    routes/
    controllers/
    services/
    validators/
    jobs/
    utils/
    tests/

shared/
  types/

scripts/
docs/

Also provide:
- README.md
- `.env.example`
- Database initialization and demo-seeding scripts.
- API documentation.
- Test instructions.
- Deployment instructions.
- Security notes and known limitations.

Adjust the structure if needed, but maintain clear separation of
concerns.
```

## 14. ACCEPTANCE CRITERIA

The application is complete only when:

1. Users can authenticate and access only permitted modules.
2. Institutions can create and submit FDR records.
3. FDR data persists in the configured database.
4. Authorized reviewers can review and decide submissions.
5. Correction and resubmission workflows work correctly.
6. Documents are private and protected by server-side authorization.
7. Maturity reminders are calculated correctly and do not generate duplicates.
8. Duplicate detection flags potential matches for review.
9. Renewal and release requests have independent statuses and histories.
10. Audit events are recorded for important operations.
11. Reports use actual authorized data.
12. All visible buttons and forms perform their intended actions.
13. Responsive layouts work on desktop, tablet, and mobile.
14. Core automated tests pass.
15. No real financial transactions are performed by the demo.
16. No real secrets or privileged production credentials are included in the source code.

## 15. INSTRUCTIONS FOR GOOGLE AI STUDIO

Start by inspecting the available development environment and determining which files and tools you can actually create or modify.

Then:
1. Present a concise architecture and implementation plan.
2. Create the project structure.
3. Implement the application in small, testable stages.
4. Generate complete source files rather than isolated snippets.
5. Keep frontend and backend types and API contracts consistent.
6. Run the available tests and fix errors.
7. Explain any required external setup or credentials.
8. Clearly identify any feature that is simulated, incomplete, or dependent on an unavailable external service.

Do not claim a feature is implemented merely because its UI exists. Verify its actual behavior.

If the full application exceeds the available generation or execution limits, prioritize a secure, working vertical slice: authentication, institution dashboard, FDR creation, database persistence, document upload, verification, and audit logging. Then continue with the remaining modules without replacing working code unnecessarily.

**Final goal:** Deliver a secure, maintainable, full-stack FDR Management Portal prototype that demonstrates a complete institutional FDR workflow, meaningful automation, transparent verification, and strong security practices suitable for an SIH presentation and further development.